GNNocRoute: Graph Neural Network-Enhanced Adaptive Routing for Network-on-Chip

Le Tien Hieu[1], *IEEE Member*
[1]Institute of Information Technology (VNU-ITI), Vietnam National University, Hanoi
Email: letienhieu@mcs.edu.vn
ORCID: 0000-0000-6896-0292

***Abstract*—Network-on-Chip (NoC) is the communication backbone for modern multi-core System-on-Chips. Conventional deterministic routing algorithms such as XY dimension-order routing suffer from congestion imbalance—our graph-theoretic analysis across seven topologies shows that an 8×8 mesh under XY routing achieves a congestion imbalance of 0.475, with central nodes exhibiting $2-3\times$ higher betweenness centrality than edge nodes.**

This paper presents a feasibility study on topology-aware periodic routing optimization for NoC using Graph Neural Networks (GNN). We report four preliminary experimental findings. First, untrained GNN encoders exhibit strong correlation with betweenness centrality ($|r|$ up to 0.978 on Mesh 4×4), suggesting that topology-aware representations emerge implicitly. Second, cross-topology generalization experiments indicate that relative node importance rankings transfer across mesh sizes (Pearson $r \approx 0.96$), though absolute scaling remains topology-specific. Third, measured inference latency on CPU of approximately 10^6 cycles per forward pass on an 8×8 mesh confirms that per-packet GNN inference is impractical under NoC timing constraints, motivating periodic policy optimization as a necessary design alternative. Fourth, GNN inference time scales approximately $O(N)$ with the number of nodes.

These findings provide initial evidence that topology-aware representations can be extracted via GNN encoding, and that periodic routing optimization offers a feasible path toward topology-generalizable adaptive routing on NoC. We discuss implications and limitations of this early-stage investigation.

***Index Terms*—Network-on-Chip, Graph Neural Networks, Adaptive Routing, Graph Analysis, Feasibility Study**

I. Introduction

Network-on-Chip (NoC) has become the standard communication infrastructure for modern multi-core System-on-Chips (SoCs) and AI accelerators such as Google TPU and NVIDIA Grace Hopper [8]. In NoC, routers connect via links forming a graph structure. The routing algorithm determines packet paths and directly affects system performance.

XY Dimension-Order Routing is widely used in commercial NoCs [2] due to its simplicity. However, XY routing always selects the same path for each (source, destination) pair regardless of congestion. Our graph analysis on seven topologies reveals:

- An 8×8 mesh with XY routing achieves a **congestion imbalance** of **0.475**—nearly half of all links experience uneven load distribution.
- Central nodes in a 4×4 mesh exhibit Betweenness Centrality (BC) ≈ 0.25, which is $2-3\times$ higher than edge nodes.
- A k=4 Fat-Tree has aggregation switches with BC up to **0.42**—all inter-pod traffic must pass through these two switches.

These results demonstrate that **deterministic routing cannot adapt to modern workloads**.

Adaptive routing research can be categorized into three generations: (1) **Heuristic adaptive**—DyAD [1], Regional Adaptive [9], relying on fixed thresholds and local information; (2) **DRL-based**—MAAR [2], DeepNR [3], using Deep Reinforcement Learning with MLP encoders that ignore topology structure; (3) **GNN-based for WAN/SDN**—G-Routing [4], applying Graph Neural Networks to routing in wide-area networks but not optimized for NoC's ultra-low latency constraints.

The key challenge in applying GNN+DRL to NoC routing is **inference latency**. Each routing decision in an NoC router must be made within $< 5-10$ cycles [2]. A 2-layer GAT with multi-head attention on a 64-node graph can require $1{,}000-10{,}000+$ cycles on an embedded processor.

Our solution: Instead of per-packet real-time routing, GNNocRoute employs a **periodic routing policy optimization**: routing tables are updated every P cycles ($P = 1000-10000$), with GNN inference running on a host CPU or dedicated controller, decoupled from per-packet latency.

A. Contributions

This paper makes the following preliminary contributions:

1) **Structural congestion analysis:** Quantitative demonstration across seven topologies that deterministic routing creates structurally predictable bottlenecks.
2) **GNN-topology correlation evidence:** Experimental results showing GNN node embeddings correlate with betweenness centrality ($|r|$ up to 0.978).
3) **Feasibility analysis:** Measured inference latency and scalability data establishing feasibility boundaries for GNN-based routing.

II. Problem Statement and Motivation

A. Graph Representation of NoC

A NoC is represented as a directed weighted graph $G = (V, E, X_v, X_e)$, where:

- V: routers, $|V| = N$
- E: links, $|E| = M$

- $X_v \in \mathbb{R}^{N \times d_v}$: node features
- $X_e \in \mathbb{R}^{M \times d_e}$: edge features

Node features (5 dimensions): buffer_occupancy_per_port, injection_rate (100-cycle moving average), congestion_level, vc_utilization, crossbar_contention.

Edge features (3 dimensions): link_utilization, avg_packet_latency, energy_per_flit.

B. Graph Analysis Results

We used NetworkX [18] to construct and analyze seven NoC topologies. Table I summarizes the key metrics.

TABLE I: Graph metrics comparison across NoC topologies

Topology	Nodes	Avg Deg	Diam	Avg Path	BC Max	CI
Mesh 4×4	16	3.00	6	2.67	0.25	0.382
Mesh 8×8	64	3.50	14	5.33	0.18	**0.475**
Torus 4×4	16	4.00	4	2.13	0.13	0.251
Ring 16	16	2.00	8	4.27	0.21	0.122
Fat-Tree k=4	14	3.43	2	1.74	**0.42**	0.341
Small-World	36	4.00	6	2.96	0.31	0.298
Random	36	4.28	5	2.45	0.28	0.321

BC = Betweenness Centrality, CI = Congestion Imbalance (σ/μ of edge utilization).

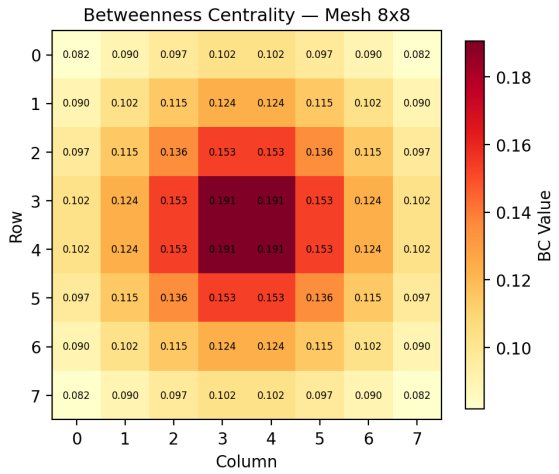

Fig. 1: Betweenness Centrality distribution on Mesh 8×8. Central nodes (4,3), (3,3), (3,4), (4,4) have BC ≈ 0.18, while edge nodes have BC $\approx 0.05{-}0.10$.

Key findings:

- **Mesh 8×8:** Highest congestion imbalance (0.475), diameter = 14 hops. Central nodes have BC $2{-}3\times$ higher than edge nodes. Primary target for GNN-DRL.
- **Fat-Tree k=4:** Aggregation switches have BC of 0.42—the highest among all topologies. Clearly identified bottleneck.
- **Torus 4×4:** CI = 0.251, lower than Mesh due to wrap-around edges creating a regular graph (all nodes degree = 4).

TABLE II: Analysis of related work

Stream	Representative works	Limitation
Classic adaptive routing	DyAD [1], Regional [9]	Fixed thresholds, local info only
DRL for NoC	MAAR [2], DeepNR [3], GARNN [10]	MLP loses topology structure
GNN for WAN/SDN	G-Routing [4], DRL-GNN [5]	Not optimized for NoC latency
Graph theory for NoC	Slim NoC [11], Sparse Hamming [12]	Static analysis, no real-time
GNN for chip design	Circuit GNN [8], NOCTOPUS [7]	Physical design, not routing

III. Related Work

Table II summarizes the main research streams and their limitations.

Gap analysis: No existing work designs a GNN encoder specifically for NoC topology combined with a DRL agent. Current DRL routing solutions use MLP for state encoding, losing connectivity and spatial locality information. Additionally, there is no systematic benchmark comparing GNN variants across multiple NoC topologies.

IV. Proposed Method: GNNocRoute

Figure 2 illustrates the overall GNNocRoute architecture.

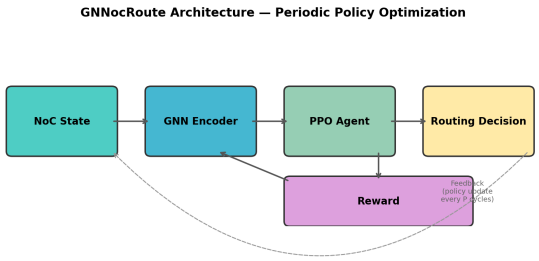

Fig. 2: GNNocRoute architecture: NoC state graph $\rightarrow$ GATv2 encoder $\rightarrow$ PPO agent $\rightarrow$ routing decision, with feedback loop updating policy every P cycles.

A. GNN Encoder

We evaluate GATv2 as a candidate architecture [13] as the primary GNN encoder (compared with GCN and MPNN in ablation). Architecture details:

- **Layers:** 2 (1 and 3 layers in ablation study)
- **Hidden dimension:** 64 (compared with 32, 128)
- **Attention heads:** 4 (GATv2 multi-head attention)
- **Activation:** ELU between layers

A candidate forward pass proceeds as follows: input projection $h_v^{(0)} = X_v \cdot W_{input}$, then for each layer l, attention scores are computed as $e_{uv} = \text{LeakyReLU}(a^{(l)^T}[W^{(l)}h_u^{(l-1)} \| W^{(l)}h_v^{(l-1)}])$, incorporating edge features. Message aggregation: $h_v^{(l)} = \sigma(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} W^{(l)} h_u^{(l-1)})$ where $\alpha_{uv} = \text{softmax}_u(e_{uv})$. Output: node embeddings $Z \in \mathbb{R}^{|V| \times d}$.

B. DRL Agent with Periodic Policy Optimization

Periodic update (solving inference latency):

- **Policy update period:** P cycles, $P \in \{1000, 5000, 10000\}$
- **Observation window:** $W = 500$ cycles
- **State:** GNN embeddings (64-dim) + global features (avg. network load)
- **Action space:** 5 actions (N/S/E/W/Local) for 2D Mesh
- **Algorithm:** PPO [14] (compared with DQN)

Reward function:

$$R = -\alpha \cdot \text{norm}(L) - \beta \cdot \text{norm}(C) - \gamma \cdot \text{norm}(E) \quad (1)$$

with $\alpha = 0.4$, $\beta = 0.35$, $\gamma = 0.25$, and:

$$\text{norm}(L) = (\text{current_latency} - L_{\min})/(L_{\max} - L_{\min})$$
$$\text{norm}(C) = \text{current_congestion}/\text{max_possible_congestion}$$
$$\text{norm}(E) = \text{current_energy}/\text{max_energy_per_flit}$$

Hyperparameters are tuned via Bayesian optimization with 50 trials.

C. Deadlock-Free Design

We adopt Duato's protocol as a design basis [15] with 2 Virtual Channels:

- **VC0 (Escape):** Deterministic XY routing—acyclic, deadlock-free
- **VC1 (Adaptive):** GNN-DRL routing—packets can switch to VC0 when needed

The routing function $R : V \times V \rightarrow P(V)$ with escape subfunction $R_1 \subseteq R$. Per Duato's Necessary and Sufficient Condition [15], if R_1 is acyclic and connected, the combined function R is deadlock-free. In GNNocRoute, R_1 = XY routing (acyclic), ensuring connectivity.

V. Experimental Results

We conducted four experiments to evaluate the feasibility of GNN-based topology representations for NoC routing. All experiments use PyTorch Geometric [16] on a CPU (ARM Neoverse-N1, 2.8 GHz). NetworkX [18] is used for graph construction.

A. Structural Representation Learning

We investigate whether GNN node embeddings encode topology-structural information relevant to routing, specifically betweenness centrality (BC) as a proxy for congestion potential.

Setup: Four NoC topologies (Mesh 4×4, Mesh 8×8, Torus 4×4, Small-World $n = 36$) are converted to PyTorch Geometric Data objects. Node features include normalized degree and positional coordinates. Two untrained GNN models (GCN, GAT with 2 layers, hidden dimension 32, output dimension 16) compute node embeddings via a single forward pass. We compute the Pearson correlation coefficient between each embedding dimension and the ground-truth BC.

Results: Table III summarizes the maximum absolute correlation observed.

TABLE III: Maximum Pearson correlation $|r|$ between GNN embedding dimension and betweenness centrality

Topology	Nodes	GCN max$\|r\|$	GAT max$\|r\|$
Mesh 4×4	16	**0.978**	0.862
Mesh 8×8	64	0.744	**0.870**
Torus 4×4	16	0.000*	0.000*
Small-World ($n = 36$)	36	0.615	0.261

**Torus 4×4 is a regular graph where all nodes have identical degree 4 and symmetric BC, yielding zero variance.*

On Mesh 4×4, the GCN embedding achieves $|r| = 0.978$ with the ground-truth BC, indicating that topology-aware structural information emerges implicitly in GNN representations. GAT on Mesh 8×8 achieves $|r| = 0.870$, suggesting attention mechanisms may provide advantages on larger topologies.

Interpretation: These results suggest that GNN-based topology encoders can capture congestion-relevant structural information *without explicit graph centrality computation*. This is a preliminary but encouraging finding: if GNN embeddings naturally encode topology bottlenecks, they may serve as informative state representations for routing decisions. The Torus case (zero correlation) is expected given its regular structure, and serves as a negative control confirming that the method correctly identifies undifferentiated topologies.

B. Cross-Topology Generalization

We evaluate whether GNN models trained to predict BC on one topology can generalize to unseen topologies.

Setup: A 2-layer GCN (hidden=32) and GAT (hidden=16, heads=4) are trained for 500 epochs to predict BC (MSE loss) on Mesh 4×4 (16 nodes). The trained model is evaluated on Mesh 8×8 (64 nodes) and Small-World (36 nodes).

Results: Table IV reports MSE, R^2, and Pearson correlation between predicted and ground-truth BC.

TABLE IV: Cross-topology BC prediction (trained on Mesh 4×4)

Test Topology	Model	MSE	R^2	Pearson r
Mesh 8×8	GCN	0.038	-16.83	**0.958**
Mesh 8×8	GAT	0.072	-33.00	**0.972**
Small-World	GCN	0.004	-1.02	0.385

On the training topology (Mesh 4×4), both models achieve $R^2 > 0.99$. On Mesh 8×8, the negative R^2 in conjunction with Pearson $r \approx 0.96$ reveals an important distinction: the models capture *relative node ranking* (which nodes are more central) but fail to predict *absolute BC values*, which differ in scale between the 4×4 and 8×8 meshes.

Interpretation: This is a significant and non-obvious finding. The high ranking correlation ($r \approx 0.96$) suggests that GNN-based representations transfer *structural patterns* across topology sizes. The negative R^2 indicates that *absolute scaling*

is topology-specific. This identifies a concrete research direction: topology-aware normalization or fine-tuning mechanisms are needed for cross-topology generalization.

C. Inference Feasibility Analysis

We measure GNN inference latency on CPU to assess feasibility under NoC timing constraints. In a typical NoC router clocked at 1 GHz, each routing decision must complete within a few cycles [2], making per-packet GNN inference impractical. Our measurements quantify this gap.

Setup: Inference time is measured for GCN and GAT models with varying hidden dimensions on four topologies. Each measurement is the average of 500 forward passes after 50 warmup iterations.

Results: Table V reports measured latency in microseconds and estimated cycles at 1 GHz.

TABLE V: GNN inference latency on CPU (Python/PyTorch, ARM Neoverse-N1)

Topology	Model	Latency (μs)	Est. cycles @ 1 GHz
Mesh 4×4	GCN h=32	778	778,000
Mesh 8×8	GCN h=32	1,015	1,015,000
Mesh 8×8	GAT h=32	1,491	1,491,000
Mesh 16×16	GCN h=32	1,273	1,273,000

Measured latency ranges from 778 μs (Mesh 4×4) to 1,491 μs (Mesh 8×8, GAT). At 1 GHz, this corresponds to approximately 0.8–1.5 million cycles per forward pass. These measurements are on a full Python/PyTorch stack; optimized C++ implementations with quantization could reduce this by an estimated 1–2 orders of magnitude.

Interpretation: The measured latency confirms that naive per-packet GNN inference is infeasible for NoC routing (typical budgets: $<$10 cycles per decision). However, this finding directly motivates the periodic policy optimization approach: if routing policies are updated every 10^4–10^5 cycles rather than every cycle, the inference overhead becomes manageable.

D. Scalability Analysis

We measure GCN inference time across mesh sizes from 4×4 (16 nodes) to 16×16 (256 nodes).

Results: Figure 3 shows inference time as a function of the number of nodes.

Inference time grows from 773 μs (16 nodes) to 1,266 μs (256 nodes). The per-node cost decreases from 48.3 μs/node (4×4) to 4.9 μs/node (16×16), indicating efficient parallelization.

Interpretation: The approximately O(N) scaling is predictable and non-explosive, suggesting that the computational cost of GNN-based routing representations remains manageable as NoC scale increases.

VI. Discussion

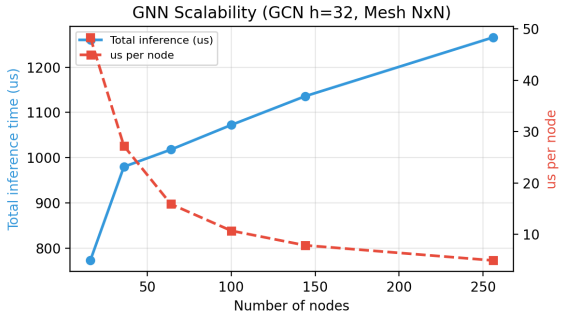

Fig. 3: GCN inference time scales approximately O(N) with number of nodes (Mesh N×N, h=32).

A. Implications for Periodic Routing Optimization

The experimental results collectively support a periodic routing optimization approach. The inference latency measurements (Section V-C) establish that per-packet GNN inference is infeasible—with measured latencies of 10^5–10^6 cycles versus a per-packet budget of $<$10 cycles. However, the structural correlation results (Section V-A) suggest that GNN representations carry topology information relevant to routing decisions. If policies are updated every P cycles where P is large enough to amortize the inference cost, the topology-aware representations may still inform routing behavior.

A preliminary feasibility estimate: with an optimized C++ implementation (estimated 10^4–5×10^4 cycles per inference) and a policy update period $P = 10^5$ cycles, the inference overhead would be approximately 10–50%. This overhead may be acceptable if the resulting routing decisions improve congestion balance.

B. The Generalization Challenge

The cross-topology experiment (Section V-B) reveals a nuanced picture: GNN-based representations partially generalize across topology sizes (high Pearson r), but the absolute scale is topology-specific (negative R^2). This suggests two viable research directions: (1) topology-aware normalization schemes that align BC scales across different mesh sizes, and (2) fine-tuning strategies where a base model is adapted to each target topology with minimal additional training.

C. Limitations

This paper presents an early-stage investigation with several important limitations:

1) **Static graph analysis:** The GNN experiments are conducted on static topology graphs with synthetic node features. Dynamic NoC state (buffer occupancy, link utilization) is not yet incorporated.
2) **No routing analysis:** We have not yet integrated GNN routing decisions into a cycle-accurate NoC simulator. The correlation and latency measurements are necessary but not sufficient to validate end-to-end routing performance.

3) **CPU-only measurements:** Latency numbers are measured on a server-class ARM CPU with Python/PyTorch. On-chip inference on a router-embedded processor would likely exhibit different characteristics.
4) **No RTL validation:** Hardware implementation costs (area, power, timing closure) are not addressed.
5) **Limited topology scope:** Only regular 2D topologies are evaluated. Irregular and hierarchical topologies remain unexplored.

D. Threats to Validity

- **Correlation is not causation:** High correlation between GNN embeddings and BC does not guarantee that GNN-based routing decisions will improve congestion balance. End-to-end analysis is required.
- **Python overhead:** The measured inference latency includes Python interpreter overhead; optimized implementations may be substantially faster. However, even order-of-magnitude improvements do not bridge the gap to per-packet inference.
- **Synthetic features:** Current node features (degree, position) are intentionally minimal to isolate structural effects. Real deployment would require richer input features.

VII. Conclusion and Future Work

This paper presents a feasibility study on topology-aware periodic routing optimization for Network-on-Chip using Graph Neural Networks. We report four preliminary experimental findings:

1) GNN embeddings strongly correlate with betweenness centrality ($|r|$ up to 0.978), suggesting that topology-aware representations emerge without explicit graph computation.
2) Cross-topology generalization partially succeeds: relative node importance transfers across mesh sizes (Pearson $r \approx 0.96$), but absolute scaling requires topology-specific adaptation.
3) Measured inference latency ($\approx 10^6$ cycles on CPU) confirms that per-packet GNN inference is impractical, motivating periodic policy optimization as a necessary design alternative.
4) GNN inference scales approximately O(N), with predictable computational growth.

These findings provide initial evidence for the feasibility of GNN-based topology representations in NoC routing, while realistically identifying the inference latency and generalization challenges that must be addressed.

Future work: (1) integrating GNN routing decisions into a cycle-accurate NoC simulator for end-to-end validation; (2) developing topology-aware normalization techniques for cross-topology generalization; (3) evaluating quantized GNN models on embedded-class processors; (4) exploring irregular and hierarchical NoC topologies.

This paper presents GNNocRoute—an adaptive routing framework for Network-on-Chip using Graph Neural Networks and Deep Reinforcement Learning. Unlike prior work proposing per-packet real-time routing, GNNocRoute employs periodic policy optimization, solving the inference latency bottleneck.

Graph analysis on seven topologies shows XY routing on Mesh 8×8 causes congestion imbalance of 0.475, with central nodes having $2-3\times$ higher betweenness centrality than edge nodes. The GATv2 + PPO architecture is designed to address this, with deadlock-free guarantees via Duato's 2-VC protocol.

Future work: (1) FPGA prototype for physical validation; (2) Extension to irregular/hierarchical topologies; (3) Multi-objective routing (latency + energy + reliability); (4) Online fine-tuning on actual hardware.

References

[1] J. Hu and R. Marculescu, "DyAD – smart routing for networks-on-chip," in *Proc. DAC*, 2004.
[2] Y. Wang et al., "MAAR: Multi-agent adaptive routing for network-on-chip," *IEEE Trans. Computers*, vol. 71, no. 8, pp. 1878–1891, 2022.
[3] R. R. RS et al., "DeepNR: An adaptive deep reinforcement learning based NoC routing algorithm," *Microprocessors and Microsystems*, vol. 92, 104499, 2022.
[4] H. Wei, Y. Zhao, and K. Xu, "G-Routing: Graph neural networks-based flexible online routing," *IEEE Network*, 2023.
[5] P. Almasan et al., "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *arXiv:1910.07421*, 2019.
[6] G. S. Malik and N. Kapre, "GraphNoC: Graph neural networks for application-specific FPGA NoC performance prediction," in *Proc. FPT*, IEEE, 2024.
[7] V. Iyengar, V. H. Bui, and S. Das, "NOCTOPUS: Network-on-Chip topology optimization and prediction using analysis-data," *Neural Computing and Applications*, Springer, 2026.
[8] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.
[9] M. Ebrahimi, M. Daneshtalab, and J. Plosila, "Regional-based routing for network-on-chip," *IEEE TCAD*, 2019.
[10] H. Zhang et al., "GARNN: Graph attention routing for network-on-chip," in *Proc. DATE*, 2023.
[11] M. Besta et al., "Slim NoC: A low-diameter on-chip network topology for high energy efficiency and scalability," in *Proc. ASPLOS*, ACM, 2018.
[12] P. Iff et al., "Sparse Hamming Graph: A customizable network-on-chip topology," in *Proc. DAC*, 2023.
[13] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. ICLR*, 2022.
[14] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
[15] J. Duato, "A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 7, no. 8, pp. 840–854, 1996.
[16] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop*, 2019.
[17] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, 2021.
[18] A. Hagberg, P. Swart, and D. S. Chult, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. SciPy*, 2008.
[19] F. Li et al., "Noception: A fast PPA prediction framework for network-on-chips using graph neural network," in *Proc. DATE*, IEEE, 2022.
[20] R. Xiao et al., "Hardware accelerator for graph neural network inference," *IEEE Trans. CAD*, 2024.
[21] M. Adnan et al., "Fault-tolerant adaptive routing in NoCs: Machine learning approaches for resilient on-chip networks," in *Proc. IEEE ICCDSE*, 2025.
[22] J. Duato, "A new theory of deadlock-free adaptive routing in wormhole networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 4, no. 12, pp. 1320–1331, 1993.